MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH & STUDIES

SCHOOL OF COMPUTER APPLICATIONS

PRACTICAL FILE

DATA STRUCTURES & ALGORITHMS

ACADEMIC SESSION: 2026-27

SUBMITTED TO

Mr. Sachin Sharma

Faculty / Assistant Professor

School of Computer Applications

MRIIRS

SUBMITTED BY

Mehak Garg

Roll No. : 26MCAP101N053

Course : MCA

Semester : 1

INDEX OF EXPERIMENTS

S. No.	Name of the Program	Bloom level with Cognitive process
1.	Write a program to perform various operations in the list: - Insertion - Deletion - Display	Apply (Using basic list operations)/ L3
2	Write a Program to sort the list using: - Quick Sort - Merge Sort	Apply (Sorting algorithm) / L3
3	Write a program to perform all operations: - For the Stack using an Array. - For the Queue using an Array. - For the Circular Queue using an Array. - Implement browser history using stacks.	Apply (Array manipulation, stack & queue operations) / L3
4	Write a program to evaluate the Postfix Notation.	Apply (Expression evaluation) / L3
5	Write a program to convert infix notation into Postfix Notation.	Apply (Expression conversion) / L3
6	Write a program to perform the operations of the linked list: - Insertion - Deletion	Apply (Linked list operation)/ L3

	- Display	
7	Write a program to perform the operations of the Circular linked list: - Insertion - Deletion - Display	Apply (Circular linked list) / L3
8	Write a program to perform all operations: - For Stack using Linked List. - For Queue using Linked List - For Circular Queue using Linked List	Apply (Linked list manipulation)/ L3
9	Write a program to perform concatenation of two linked lists and Implement Polynomial Representation and Addition using Linked Lists	Apply (Linked list manipulation)/ L3
10	Write a program to perform the operations of the Double linked list: - Insertion - Deletion - Display	

EXPERIMENT 1: Write a program to perform various operations in the list:

- Insertion
- Deletion
- Display

Input-

```
list1 = [10, 20, 30, 40]

while True:
    print("1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        n = int(input("Enter element: "))
        list1.append(n)
        print("Element inserted")

    elif ch == 2:
        n = int(input("Enter element to delete: "))
        if n in list1:
            list1.remove(n)
            print("Element deleted")
        else:
            print("Element not found")

    elif ch == 3:
        print("List is:", list1)

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

Output-

```
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter element: 50
Element inserted
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3
List is: [10, 20, 30, 40, 50]
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 2
Enter element to delete: 30
Element deleted
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3
List is: [10, 20, 40, 50]
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 4
```

EXPERIMENT 2: Write a Program to sort the list using:

- Quick Sort
- Merge Sort

Quick Sort:

Input-

```
#quick sort

def quick_sort(a):
    if len(a) <= 1:
        return a

    p = a[0]
    left = []
    right = []

    for i in a[1:]:
        if i < p:
            left.append(i)
        else:
            right.append(i)

    return quick_sort(left) + [p] + quick_sort(right)
```

```
a = [40, 10, 30, 20, 50]
print("Original list:", a)

a = quick_sort(a)
print("Sorted list:", a)
```

Output-

```
Original list: [40, 10, 30, 20, 50]
Sorted list: [10, 20, 30, 40, 50]
```

Merge Sort:

Input-

```
#merge sort

def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []

    while len(left) > 0 and len(right) > 0:
        if left[0] < right[0]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))
```

```
    result = result + left + right
    return result


a = [40, 10, 30, 20, 50]
print("Original list:", a)

a = merge_sort(a)
print("Sorted list:", a)
```

Output-

```
Original list: [40, 10, 30, 20, 50]
Sorted list: [10, 20, 30, 40, 50]
```

EXPERIMENT 3: Write a program to perform all operations:

- For the Stack using an Array.
- For the Queue using an Array.
- For the Circular Queue using an Array.
- Implement browser history using stacks.

1. Stack Using an Array

Input-

```
#stack using array
stack = []

while True:
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        n = int(input("Enter element: "))
        stack.append(n)
        print("Element pushed")

    elif ch == 2:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Deleted element:", stack.pop())

    elif ch == 3:
        print("Stack:", stack)

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

Output-

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 30
Element pushed
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack: [30]
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Deleted element: 30
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4
```

2. Queue Using an Array

Input-

```python
#queue using array

queue = []

while True:
    print("1. Front")
    print("2. Rear")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        n = int(input("Enter element: "))
        queue.append(n)
        print("Element inserted")
```

```python
elif ch == 2:
    if len(queue) == 0:
        print("Queue is empty")
    else:
        print("Deleted element:", queue.pop(0))

elif ch == 3:
    print("Queue:", queue)

elif ch == 4:
    break

else:
    print("Invalid choice")
```

Output-

```
1. Front
2. Rear
3. Display
4. Exit
Enter choice: 1
Enter element: 30
Element inserted
1. Front
2. Rear
3. Display
4. Exit
Enter choice: 3
Queue: [30]
1. Front
2. Rear
3. Display
4. Exit
Enter choice: 2
Deleted element: 30
```

```
1. Front
2. Rear
3. Display
4. Exit
Enter choice: 4
```

3. Circular Queue Using an Array

Input-

```
#circular queue using array


size = 5
queue = [0] * size
front = -1
rear = -1

while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        if (rear + 1) % size == front:
            print("Queue is full")
```

```
        else:
            n = int(input("Enter element: "))

            if front == -1:
                front = 0
                rear = 0
            else:
                rear = (rear + 1) % size

            queue[rear] = n
            print("Element inserted")

    elif ch == 2:
        if front == -1:
            print("Queue is empty")
        else:
            print("Deleted element:", queue[front])

            if front == rear:
```

```
                front = -1
                rear = -1
            else:
                front = (front + 1) % size

    elif ch == 3:
        if front == -1:
            print("Queue is empty")
        else:
            i = front
            while True:
                print(queue[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % size
            print()

    elif ch == 4:
        break
```

```
    else:
        print("Invalid choice")
```

Output-

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 10
Element inserted
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 20
Element inserted
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
```

```
Enter element: 30
Element inserted
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 20
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
```

```
Enter element: 40
Element inserted
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 50
Element inserted
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 60
Element inserted
1. Enqueue
2. Dequeue
3. Display
```

```
4. Exit
Enter choice: 3
30 40 50 60
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 4
```

4. Browser History Using Stack

Input-

```python
#Browser history using stack

last = []

while True:
    print("1. Visit Website")
    print("2. Go Back")
    print("3. Show History")
    print("4. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        website = input("Enter website name: ")
        last.append(website)
        print("Website opened:", website)
```

```python
elif ch == 2:
    if len(last) > 1:
        last.pop()
        print("Back to:", last[-1])
    else:
        print("No previous page available")

elif ch == 3:
    print("Browser History:", last)

elif ch == 4:
    break

else:
    print("Invalid choice")
```

Output-

```
1. Visit Website
2. Go Back
3. Show History
4. Exit
Enter your choice: 1
Enter website name: google
Website opened: google
1. Visit Website
2. Go Back
3. Show History
4. Exit
Enter your choice: 1
Enter website name: youtube
Website opened: youtube
1. Visit Website
2. Go Back
3. Show History
4. Exit
Enter your choice: 2
```

```
Back to: google
1. Visit Website
2. Go Back
3. Show History
4. Exit
Enter your choice: 3
Browser History: ['google']
1. Visit Website
2. Go Back
3. Show History
4. Exit
Enter your choice: 2
No previous page available
1. Visit Website
2. Go Back
3. Show History
4. Exit
Enter your choice: 4
```

EXPERIMENT 4: Write a program to evaluate the Postfix Notation.

Input-

```
exp = input("Enter postfix expression: ")
stack = []

for i in exp.split():
    if i.isdigit():
        stack.append(int(i))
    else:
        b = stack.pop()
        a = stack.pop()

        if i == '+':
            stack.append(a + b)
        elif i == '-':
            stack.append(a - b)
        elif i == '*':
            stack.append(a * b)
        elif i == '/':
            stack.append(a / b)
```

```
print("Result:", stack.pop())
```

Output-

```
Enter postfix expression: 2 3 + 5 *
Result: 25
```

EXPERIMENT 5: Write a program to convert infix notation into Postfix Notation.

Input-

```
exp = input("Enter infix expression: ")
stack = []
postfix = ""

for i in exp:
    if i.isalnum():
        postfix = postfix + i

    elif i == '(':
        stack.append(i)

    elif i == ')':
        while stack and stack[-1] != '(':
            postfix = postfix + stack.pop()
        stack.pop()
    else:
        while stack and stack[-1] != '(':
            if (i in '+-' and stack[-1] in '+-*/') or \
               (i in '*/' and stack[-1] in '*/'):
                postfix = postfix + stack.pop()
            else:
                break
        stack.append(i)

while stack:
    postfix = postfix + stack.pop()

print("Postfix expression:", postfix)
```

Output-

```
Enter infix expression: A+B*C
Postfix expression: ABC*+
```

EXPERIMENT 6: Write a program to perform the operations of the linked list:

- Insertion
- Deletion
- Display

Input-

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))
```

```python
if ch == 1:
    n = int(input("Enter number: "))
    new = Node(n)

    if head == None:
        head = new
    else:
        temp = head
        while temp.next != None:
            temp = temp.next
        temp.next = new

    print("Inserted")

elif ch == 2:
    n = int(input("Enter number to delete: "))
    temp = head
    prev = None
```

```python
      while temp != None:
          if temp.data == n:
              if prev == None:
                  head = temp.next
              else:
                  prev.next = temp.next
              print("Deleted")
              break

          prev = temp
          temp = temp.next

      if temp == None:
          print("Number not found")

  elif ch == 3:
      temp = head
```

```python
      if temp == None:
          print("List is empty")
      else:
          while temp != None:
              print(temp.data, end=" -> ")
              temp = temp.next
          print("None")

  elif ch == 4:
      break

  else:
      print("Wrong choice")
```

Output-

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter number: 10
Inserted
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter number: 20
Inserted
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
```

```
10 -> 20 -> None
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2
Enter number to delete: 10
Deleted
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 4
```

EXPERIMENT 7: Write a program to perform the operations of the Circular linked list:

- Insertion
- Deletion
- Display

Input-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
```

```
        n = int(input("Enter element: "))
        new = Node(n)

        if head is None:
            head = new
            new.next = head
        else:
            temp = head
            while temp.next != head:
                temp = temp.next

            temp.next = new
            new.next = head

        print("Element inserted")

    elif ch == 2:
        if head is None:
            print("List is empty")
```

```
    elif head.next == head:
        print("Deleted element:", head.data)
        head = None
    else:
        n = int(input("Enter element to delete: "))

        if head.data == n:
            temp = head
            while temp.next != head:
                temp = temp.next

            print("Deleted element:", head.data)
            head = head.next
            temp.next = head
        else:
            temp = head
            while temp.next != head and temp.next.data != n:
                temp = temp.next
```

```
                if temp.next == head:
                    print("Element not found")
                else:
                    print("Deleted element:", temp.next.data)
                    temp.next = temp.next.next

    elif ch == 3:
        if head is None:
            print("List is empty")
        else:
            temp = head
            while True:
                print(temp.data, end=" ")
                temp = temp.next
                if temp == head:
                    break
            print()
```

```
    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

Output-

```
1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 1
Enter element: 10
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 1
Enter element: 20
Element inserted

1. Insertion
2. Deletion
```

```
3. Display
4. Exit
Enter choice: 1
Enter element: 30
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 3
10 20 30

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 2
Enter element to delete: 20
```

```
Deleted element: 20

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 3
10 30

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 4
```

EXPERIMENT 8: Write a program to perform all operations:

- For Stack using Linked List.
- For Queue using Linked List
- For Circular Queue using Linked List

1. Stack Using Linked List:

Input-

main.py using ll

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

top = None

while True:
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        n = int(input("Enter element: "))
```

```
        new = Node(n)
        new.next = top
        top = new
        print("Element inserted")

    elif ch == 2:
        if top is None:
            print("Stack is empty")
        else:
            print("Deleted element:", top.data)
            top = top.next

    elif ch == 3:
        temp = top
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()
```

```
elif ch == 4:
    break

else:
    print("Invalid choice")
```

Output-

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 29
Element inserted

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter element: 32
Element inserted
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
32 29

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Deleted element: 32

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
```

```
Deleted element: 29

1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4
```

2. Queue Using Linked List:

Input-

```
#Queue Using Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

front = None
rear = None

while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        n = int(input("Enter element: "))
```

```
        new = Node(n)

        if front is None:
            front = new
            rear = new
        else:
            rear.next = new
            rear = new

        print("Element inserted")

    elif ch == 2:
        if front is None:
            print("Queue is empty")
        else:
            print("Deleted element:", front.data)
            front = front.next

            if front is None:
```

```
                rear = None

    elif ch == 3:
        temp = front
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

Output-

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 20
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 39
Element inserted

1. Enqueue
2. Dequeue
3. Display
```

```
4. Exit
Enter choice: 3
20 39

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 20

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 4
```

3. Circular Queue Using Linked List

Input-

```python
#Circular Queue Using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

front = None
rear = None

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        n = int(input("Enter element: "))
        new = Node(n)

        if front is None:
            front = new
            rear = new
            rear.next = front
        else:
            rear.next = new
            rear = new
            rear.next = front

        print("Element inserted")

    elif ch == 2:
        if front is None:
            print("Queue is empty")
        else:
            print("Deleted element:", front.data)
```

```
        if front == rear:
            front = None
            rear = None
        else:
            front = front.next
            rear.next = front

elif ch == 3:
    if front is None:
        print("Queue is empty")
    else:
        temp = front
        while True:
            print(temp.data, end=" ")
            temp = temp.next

            if temp == front:
                break
        print()
```

```
elif ch == 4:
    break

else:
    print("Invalid choice")
```

Output-

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 28
Element inserted

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter element: 26
Element inserted

1. Enqueue
2. Dequeue
3. Display
```

```
4. Exit
Enter choice: 3
28 26

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 28

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 26

1. Enqueue
```

```
2. Dequeue
3. Display
4. Exit
Enter choice: 4
```

EXPERIMENT 9: Write a program to perform concatenation of two linked lists and Implement Polynomial Representation and Addition using Linked Lists.

1. Concatenation of Two Linked Lists:

Input-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


def display(head):
    temp = head
    while temp is not None:
        print(temp.data, end=" ")
        temp = temp.next
    print()


head1 = None
head2 = None
```

```
n = int(input("Enter number of elements in first list: "))
for i in range(n):
    x = int(input("Enter element: "))
    new = Node(x)

    if head1 is None:
        head1 = new
        temp = head1
    else:
        temp.next = new
        temp = new

n = int(input("Enter number of elements in second list: "))
for i in range(n):
    x = int(input("Enter element: "))
    new = Node(x)

    if head2 is None:
        head2 = new
        temp = head2
    else:
        temp.next = new
        temp = new

if head1 is None:
    head1 = head2
else:
    temp = head1
    while temp.next is not None:
        temp = temp.next
    temp.next = head2

print("Concatenated list:")
display(head1)
```

Output-

```
Enter number of elements in first list: 4
Enter element: 10
Enter element: 20
Enter element: 30
Enter element: 40
Enter number of elements in second list: 2
Enter element: 60
Enter element: 80
Concatenated list:
10 20 30 40 60 80
```

2. Polynomial Representation and Addition Using Linked Lists:

Input-

```
class Node:
    def __init__(self, coeff, power):
        self.coeff = coeff
        self.power = power
        self.next = None


def insert(head, coeff, power):
    new = Node(coeff, power)

    if head is None:
        return new

    temp = head
    while temp.next is not None:
        temp = temp.next
    temp.next = new
    return head


def display(head):
    temp = head

    while temp is not None:
        print(str(temp.coeff) + "x^" + str(temp.power), end=" ")
        temp = temp.next
        if temp is not None:
            print("+", end=" ")

    print()


def add(p1, p2):
    result = None
```

```
    while p1 is not None and p2 is not None:
        if p1.power == p2.power:
            c = p1.coeff + p2.coeff
            if c != 0:
                result = insert(result, c, p1.power)
            p1 = p1.next
            p2 = p2.next

        elif p1.power > p2.power:
            result = insert(result, p1.coeff, p1.power)
            p1 = p1.next

        else:
            result = insert(result, p2.coeff, p2.power)
            p2 = p2.next

    while p1 is not None:
        result = insert(result, p1.coeff, p1.power)
        p1 = p1.next
```

```
    while p2 is not None:
        result = insert(result, p2.coeff, p2.power)
        p2 = p2.next

    return result


p1 = None
p2 = None

p1 = insert(p1, 3, 2)
p1 = insert(p1, 2, 1)
p1 = insert(p1, 4, 0)

p2 = insert(p2, 5, 2)
p2 = insert(p2, 3, 1)
p2 = insert(p2, 2, 0)
```

```
print("First polynomial:")
display(p1)

print("Second polynomial:"
display(p2)

result = add(p1, p2)

print("Addition:")
display(result)
```

Output-

```
First polynomial:
3x^2 + 2x^1 + 4x^0
Second polynomial:
5x^2 + 3x^1 + 2x^0
Addition:
8x^2 + 5x^1 + 6x^0
```

EXPERIMENT 10: Write a program to perform the operations of the Double linked list:

- Insertion
- Deletion
- Display

Input-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


head = None

while True:
    print("\n1. Insertion")
    print("2. Deletion")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
```

```
    n = int(input("Enter element: "))
    new = Node(n)

    if head is None:
        head = new
    else:
        temp = head
        while temp.next is not None:
            temp = temp.next

        temp.next = new
        new.prev = temp

    print("Element inserted")

elif ch == 2:
    if head is None:
        print("List is empty")
    else:
```

```
    n = int(input("Enter element to delete: "))
    temp = head

    while temp is not None and temp.data != n:
        temp = temp.next

    if temp is None:
        print("Element not found")
    else:
        if temp.prev is None:
            head = temp.next
        else:
            temp.prev.next = temp.next

        if temp.next is not None:
            temp.next.prev = temp.prev

        print("Element deleted")
```

```
elif ch == 3:
    if head is None:
        print("List is empty")
    else:
        temp = head
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

elif ch == 4:
    break

else:
    print("Invalid choice")
```

Output-

```
1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 1
Enter element: 10
Element inserted

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 1
Enter element: 20
Element inserted

1. Insertion
2. Deletion
3. Display
```

```
4. Exit
Enter choice: 3
10 <-> 20 <-> None

1. Insertion
2. Deletion
3. Display
4. Exit
Enter choice: 4
```